Genericness Audit — CompanyIQ main app + Framework Builder V2

Scope of this audit. CompanyIQ scores companies against a user-authored framework. Framework Builder V2 turns an intake (topic, synonyms, sub-areas, sensitivity preferences) into a *hardened* framework: it discovers terminology, mines retrieval query terms and topic synonyms, and attaches anti-inference rules and negative keywords. The builder is meant to be **topic-agnostic** — the same code should build an equally good framework for AI governance, climate, modern slavery, nature/biodiversity, or any other subject. This document audits the main app and the builder for elements that are hardcoded to one topic (the AI/ML/governance framework the product was first built around) or to one company, and records how each was handled.

This audit accompanies the changes on branch `feat/generic-framework-hardening`. It covers **Goal A** (this audit + the refactors it drove) and documents the two build-time hardening changes delivered alongside it (**Goal B / "Change #1"**).

The guiding principle throughout: *derive topic-specific behaviour from the framework's own topic/intake at build time; never bake one topic's vocabulary into shared builder logic, prompts, or scoring.* Where a construct is genuinely multi-topic illustration (helping the LLM by example) rather than a hidden gate, it is left in place and flagged as reviewed.

---

1. Summary of findings

#	Location	Hardcoded element found	Verdict	How handled
A1	`server/lib/anti-inference.ts` (Layer-2 training family rule)	Example clause used the literal `'we invest in AI training'` — an AI-topic string baked into a rule applied to *every* framework's training/reskilling measures	**FIXED**	Templated on `{TOPIC}`, substituted from the framework's own `topicTerm` at compose time (falls back to "the topic").
A2	`server/lib/anti-inference.ts` (Layer-1 universal rule #2)	Parenthetical examples "(Partnership on AI, OECD signatory, SBTi commitment, UN Global Compact, NZBA member, etc.)"	**REVIEWED — kept**	These are multi-topic illustrative examples (AI + climate + ESG) inside a general "alliance membership is not evidence" principle, not a topic gate. Left as-is; removing them would weaken the rule for the LLM.
A3	`server/lib/framework-v2/retrieval-query-terms.ts` (`REFERENCE_FILING_CORPUS`)	The DF gate's reference corpus was topic-agnostic but **incomplete** — it had no SEC cover-page / fee-table / exhibit boilerplate, so filing scaffolding ("filing fee", "all boxes", "computed table") was *not* recognised as filing-generic and	**FIXED / EXTENDED**	Added SEC EDGAR cover-page, check-box and EX-FILING-FEE fee-table paragraphs to the reference corpus (still topic-neutral). Backs the new sanitiser (B2).

#	Location	Hardcoded element found	Verdict	How handled
		leaked into mined terms		
A4	`server/lib/framework-v2/intake-prompt.ts`	Multi-topic examples (“for nature: TNFD, SBTN…; for climate: TCFD…; for AI governance: LLM, AGI, GPAI”)	**REVIEWED — kept**	Legitimately generic: the prompt lists several topics side-by-side as examples so the LLM generalises. No single topic is privileged in logic.
A5	`server/lib/topic-lexicon.ts`, other builder prompts	Multi-topic worked examples	**REVIEWED — kept**	Same rationale as A4 — illustrative spread across topics, not a gate.
A6	`server/lib/passage-retrieval.ts`, `analyzer.ts`, `issuer-profile.ts`, `anomaly-detection.ts`	References to company / ticker / ISIN	**REVIEWED — no change**	All occurrences are in regression-annotation comments; a non-comment grep for company-specific literals returned empty. Entity handling is already data-driven off the company record.
B1	All framework persist paths	Hardened fields (`negativeKeywords`, `antiInferenceRules`) present in intake were **not promoted** to the top-level columns the analyzer reads — so builder-pro-	**FIXED**	New idempotent, non-destructive `promoteHardeningFields()` helper wired into every persist path.

#	Location	Hardcoded element found	Verdict	How handled
		duced frameworks shipped with `null` hardening even though the intake had it (observed on the audited framework, report §E)		
B2	Build-time synonym / query-term mining	Filing boilerplate contaminated `topicSynonyms` and mined query terms for *any* topic (report §E: 11 extraneous terms)	**FIXED**	New topic-agnostic `boilerplate-hygiene` sanitiser applied at build time, plus a maintenance script to clean existing frameworks.

No element was found that could only be fixed by hardcoding this one topic. Every fix derives topic-specific behaviour from the framework's own topic/intake or from generic, topic-neutral heuristics.

---

2. Hardcoded elements found and how each was handled

A1 — Anti-inference training rule hardcoded to "AI training" — FIXED

`MEASURE_FAMILY_RULES` in `anti-inference.ts` attaches evidence-shape rules to measures by family (published-policy, partnership, quantified-metric, training, …). These rules are meant to be framework-agnostic. The training/reskilling family rule, however, contained a concrete example clause hardcoded to AI:

> *"A single-sentence mention that 'we invest in **AI** training' is insufficient…"*

Applied to a climate or modern-slavery framework, that example is off-topic and subtly anchors the LLM on AI.

Handling. The literal was changed to a `{TOPIC}` placeholder:

> *"…a single-sentence mention that 'we invest in **{TOPIC}** training' is insufficient…"*

A small pure helper `applyTopicPlaceholder(rule, topicTerm)` substitutes `{TOPIC}` with the framework's own `topicTerm` (falling back to the literal string "the topic" when none is available). `composeAntiInferenceRules(...)` gained an optional `topicTerm?: string | null` parameter and applies the placeholder to every Layer-2 family rule it emits. Both call sites in `analyzer.ts` now pass `(framework as any)?.topicTerm`. The mechanism is generic: any family rule can now carry a `{TOPIC}` token and be templated per framework, so future rules stay topic-agnostic by construction.

A2 — Universal rule alliance examples — REVIEWED, kept

Layer-1 universal rule #2 tells the LLM not to infer a commitment from mere membership of "an alliance, initiative or signatory group (Partnership on AI, OECD signatory, SBTi commitment, UN Global Compact, NZBA member, etc.)". The examples deliberately span AI, climate and ESG. This is a **principle** with multi-topic illustration, not a topic-specific gate — the rule fires for every framework regardless of topic. Genericising away the examples would make the instruction more abstract and less reliable for the model. **Left as-is**, recorded here as reviewed.

A3 — Reference filing corpus incomplete — FIXED / EXTENDED

The document-frequency gate in `retrieval-query-terms.ts` scores a candidate term by how often it appears across `REFERENCE_FILING_CORPUS`, a set of topic-neutral "generic corporate filing" paragraphs. A term that is common across unrelated filings is filing-generic, not a topic discriminator, and is dropped.

The corpus was topic-agnostic but **did not contain SEC cover-page or fee-table boilerplate**, so structural filing language ("filing fee", "all boxes", "computed table exhibit", "paid previously") scored a *low* DF and survived — exactly the contamination the audit report observed in §E. This is a genericness defect: the gate was blind to a whole class of topic-neutral filing scaffolding.

Handling. Added SEC EDGAR cover-page / check-box scaffolding and EX-FILING-FEE fee-table paragraphs to the reference corpus. They contain no topic vocabulary, so they raise the DF of filing scaffolding for **every** framework without biasing any topic. This corpus also backs the new sanitiser (B2).

A4 / A5 — Multi-topic prompt examples — REVIEWED, kept

`intake-prompt.ts`, `topic-lexicon.ts` and related builder prompts include worked examples across several topics (nature/TNFD, climate/TCFD, AI/LLM, modern slavery lifecycle). These are few-shot illustrations that help the LLM generalise the *shape* of a good answer; no single topic drives control flow or scoring. **Kept**, recorded as reviewed.

A6 — Company / ticker / ISIN references — REVIEWED, no change

Entity resolution across `passage-retrieval.ts`, `analyzer.ts`, `issuer-profile.ts` and `anomaly-detection.ts` is already driven off the company record (name / ISIN / ticker / domain fields). A grep for company-specific string literals outside comments returned nothing; the only textual references are regression-annotation comments. **No change.**

---

3. Goal B / “Change #1” deliverables (built on this audit)

Both are additive, non-blocking, fail-loud, topic-agnostic, and create-new (never mutate an original framework object in place).

B1 — Promote hardening fields on every persist path

Problem (report §E). A framework’s intake artefact can carry `negativeKeywords` and `antiInferenceRules`, but those live under `intakeArtefact` (camelCase). The analyzer reads the **top-level** columns. On the audited framework the top-level fields were `null` while the intake had them populated — so the builder was shipping *un-hardened* frameworks despite the hardening being present in the intake.

Fix. New helper `server/lib/framework-v2/promote-hardening-fields.ts`:

```
promoteHardeningFields<T>(framework, intake?) =>
  { framework: T, changed: boolean, promoted: HardeningField[], notes: string[] }
```

- `HARDENING_FIELDS = ["negativeKeywords", "antiInferenceRules"]`.
- Source of truth = the explicit `intake` argument if given, else `framework.intakeArtefact`.
- A field is promoted **only** when the top-level value is not already a non-empty array **and** the source has a non-empty array. It therefore **never overwrites** an existing top-level value (non-destructive) and is **idempotent** (a second call is a no-op).
- Returns a shallow copy; the input object is not mutated. Handles null/garbage input without throwing (fail-safe), and records `notes` for fail-loud logging.

Wiring (every persist path):

Path	File:line
Create framework (belt-and-suspenders, logged)	`server/storage.ts:412` (`createFramework`)
Update framework (defensive, when `updates.intakeArtefact` present)	`server/storage.ts:2530` (`updateFramework`)
Builder → save (produces hardened frameworks at build)	`server/routes/framework-builder-v2.ts` (`/v2/save`)
Export as full detail	`server/lib/framework-v2/export-as-full.ts:97`
Import (export builder + insert builder)	`server/lib/framework-v2/import-framework.ts:79, 139`

The builder now **produces hardened frameworks at build time**: the save path both promotes intake hardening to top level and passes through `negativeKeywords` / `antiInferenceRules` on the create call.

B2 — Topic-agnostic filing-boilerplate sanitiser (build time + backfill)

Problem (report §E). Mined `topicSynonyms` and retrieval query terms were contaminated with filing/report scaffolding (“filing fee”, “all boxes”, “computed table”, “paid previously” — 11 extraneous terms on the audited framework). At runtime such terms promote irrelevant filing material into the corpus. This happens for **any** topic, so the fix must be generic, not a blocklist of this framework’s specific contaminants.

Fix. New module `server/lib/framework-v2/boilerplate-hygiene.ts` (pure/deterministic, no LLM, no DB):

- `FILING_BOILERPLATE_PATTERNS: RegExp[]` — structural filing heuristics only (fee-table, cover-page check-boxes, exhibits, form identifiers, “pursuant to”, “incorporated by reference”, signature scaffolding). **No topic vocabulary.**
- `isFilingBoilerplateTerm(term, { topicTokens? })` — true when a term is filing scaffolding or entirely generic filler. **Protection:** any candidate that overlaps the framework’s own lexicon (`topicTokens` = topicTerm + topicSynonyms tokens) is never flagged — so a framework genuinely *about* filings/fees keeps that vocabulary.
- `sanitizeTopicTerms(terms, { topicTokens?, corpus?, dfThreshold?, skipDfGate? })` — dedupes case-insensitively (preserving first-seen order and original casing), drops boilerplate, and applies the DF gate against `REFERENCE_FILING_CORPUS`. Returns `{ kept, dropped: [{term, reason}] }` for fail-loud logging.

The design is Goodhart-resistant and generic: it removes a *class* of topic-neutral scaffolding via heuristics + a document-frequency gate, and protects the framework’s own canonical lexicon by overlap — it does **not** hardcode this topic’s contaminants. The same rules were verified to clean an unrelated (climate) term list in tests while keeping climate vocabulary.

Wiring:

Path	File:line	Notes
Builder → save (build-time clean of `topicSynonyms`)	`server/routes/framework-builder-v2.ts:1743`	Protects `topicTerm` + `adjacentTopics` names; non-fatal.
Terminology-gap miner (build-time clean of proposed synonyms)	`server/lib/framework-v2/test-drive.ts:642` (`detectTerminologyGaps`)	Protects topicTerm + existing topicSynonyms tokens; logs dropped.
Existing-framework cleanup	`server/scripts/clean-framework-synonyms.ts`	**Dry-run by default**; `--apply` to write, `--framework N`, `--include-retrieval`. Protects topicTerm + adjacentTopics. Not run against prod in this change.

> *Note on the protection set: at every call site the protected tokens are the framework's **canonical** lexicon (topicTerm + adjacentTopics/topicSynonyms), NOT the candidate list being cleaned — otherwise boilerplate could self-protect.*

---

4. Verification

- New focused unit tests, all passing:
- `server/lib/framework-v2/promote-hardening-fields.test.ts` (8 tests) — promotion, non-destructive, idempotent, precedence, empty/garbage.
- `server/lib/framework-v2/boilerplate-hygiene.test.ts` (8 tests) — flags SEC boilerplate, does NOT flag topic vocabulary, topicTokens protection, topic-agnostic on a different topic, order/casing/dedupe.
- Related existing suites re-run green after the changes: `retrieval-query-terms`, `import-framework`, `test-drive`, `corpus-boilerplate`, `corpus-hygiene`, `topic-lexicon`.
- Typecheck: client `tsc --noEmit` = 0 errors; server `tsc --noEmit` = 61 errors, **unchanged from the pre-existing baseline** (none reference the new/edited files).
- Build: `pnpm build` exits 0.

All changes are additive and non-destructive; no LLM/model configuration was changed; no code was run against the production database and nothing was deployed.

---

5. Pass 2026-10-19 — full-app agnosticism re-sweep (accompanies Issue 1 builder-stability work)

This pass was run alongside the Issue 1 build-time verdict-stability work (commit `a8302a1`). It re-sweeps `server/`, `shared/`, and `client/src/` for topic-specific literals (AI / ML / LLM / AGI / GPAI / "AI training" / governance, plus ESG / climate / nature families), company names / tickers / ISINs from the original AI-governance roster, and any hardcoded framework-id / measure-id / measure-name strings that **gate** behaviour. The goal is to separate genuine

hidden single-topic/company gates (→ FIX) from legitimate multi-topic illustrative examples and regression-annotation comments (→ REVIEWED-KEPT), per the classification already established in §1.

5.1 Greps run (verbatim)

```
# Topic acronyms / phrases
grep -rniE '\b(AI training|AGI|GPAI|frontier model|frontier AI)\b' \
  server shared client/src --include='*.ts' --include='*.tsx' -l

# "governance" used as a string-comparison / membership GATE
grep -rniE '(===|==|includes|startsWith|match|indexOf)[^;]*["'"'"']governance' \
  server shared client/src --include='*.ts' --include='*.tsx'

# AI-governance-roster company names in non-comment code
grep -rniE '\b(Microsoft|OpenAI|Anthropic|Alphabet|Nvidia|Meta Platforms|DeepMind|
Palantir|Cohere|Mistral|Hugging ?Face|Stability AI)\b' \
  server shared client/src --include='*.ts' --include='*.tsx' | grep -vE '//|\*'

# ISIN pattern (2 letters + 9 alnum + check digit)
grep -rnoE '\b[A-Z]{2}[A-Z0-9]{9}[0-9]\b' \
  server shared client/src --include='*.ts' --include='*.tsx'

# framework-id / measure-id / name string-equality gates
grep -rniE '(frameworkId|framework_id|measureId|measure_id|frameworkName|measureName)
\s*(===|==|!==|!=)\s*["'"'"']' \
  server shared client/src --include='*.ts' --include='*.tsx' | grep -vE '\.test\.'
grep -rniE '\.(title|name|slug|topic)\s*(===|==)\s*["'"'"'][A-Za-z]' \
  server shared client/src --include='*.ts' --include='*.tsx' | grep -vE '\.test\.|
__fixtures__'

# Topic/ESG literals inside string contents in non-prompt, non-test logic
grep -rniE '["'"'"'][^"'"'"']*\b(AI governance|ESG|climate change|greenhouse gas|
carbon emission|biodiversity|deforestation)\b' \
  server shared client/src --include='*.ts' --include='*.tsx' \
  | grep -vE '\.test\.|__fixtures__|intake-prompt|topic-lexicon|-prompt\.ts|/prompts/'

# Topic literals specifically inside the SCORING path
grep -rniE '["'"'"'][^"'"'"']*\b(AI|ML|LLM|AGI|GPAI|ESG|climate|biodivers|governance)
\b' \
  server/lib/analyzer.ts
```

5.2 Findings

Location	Hardcoded element	Verdict	Handling
`server/lib/framework-v2/intake-prompt.ts:55`, `server/lib/topic-lexicon.ts:133`	Multi-topic acronym examples side-by-side (nature: TNFD/SBTN/GBF…; climate: TCFD/SBTi/CDP; AI: LLM/AGI/GPAI) in LLM prompt guidance	REVIEWED-KEPT	Legitimate multi-topic *illustrative examples* that teach the model to emit all domain acronyms for **whatever** topic it is given; removing them would weaken every topic equally. Same class as §1 A4/A5.
`server/lib/framework-v2/test-drive.ts:592`	Comment `// short known terms ("ai","ml","llm","agi") match INSIDE`	REVIEWED-KEPT	Regression comment explaining a substring-matching guard; the code around it is topic-agnostic token logic, not a gate.
`server/lib/passage-retrieval.ts:1384-1391`	`categoryLower.includes("governance" / "oversight" / "board" / "strategy" / "policy" / "framework")` → inject terminology buckets	REVIEWED-KEPT	These are **universal framework-pillar category names** (TCFD/TNFD/ISSB all use Governance/Strategy/Risk/Metrics), not the AI-governance topic. The injected `terminology.*` buckets are derived at runtime from the framework's own terminology object. Topic-agnostic routing.
LLM-provider identifiers (`deepseek`, `openai`, `anthropic/claude`, `mistral`, `gemini`, `openrouter`, `moonshot/kimi`, `zhipu/glm`) across `ai-providers`,	Model/vendor names	REVIEWED-KEPT	Scoring **infrastructure** (which model to call / price), not entities being scored. Unrelated to framework topic.

Location	Hardcoded element	Verdict	Handling
`provider-resilience.ts`, `llm-usage.*`, fixtures			
ISINs (`GB0007099541` Prudential, `JP3890350006`, `FR0000121485`, …) in `issuer-profile.ts:255` and `*.test.ts`	ISIN literals	REVIEWED-KEPT	All are either **test fixtures** or a **regression-motivation comment** (R5e Prudential-plc domain bug). None are AI-governance-roster companies; the production code derives ISIN country generically.
`measureId` / `frameworkId` string comparisons across `edit-applier.ts`, `measure-design-diagnostic.ts`, `regression/harness.ts`, `framework-builder-v2.ts`, `FrameworkBuilderV2Page.tsx`	`typeof x.measureId === "string"`; `measureId !== "framework-level"`; `measureId === "(framework)"`	REVIEWED-KEPT	Type-guards and **structural sentinel** values (framework-level vs measure-level item). No topic/company meaning.
`server/lib/framework-v2/generalisation-audit.ts:40-57`	`TOPIC_TERMS` seed lexicon (biodivers/nature/climate/emission/carbon/…) + `INFRA_ACRONYMS` allow-list	REVIEWED-KEPT	This is the **anti-hardcoding detector itself** (Detector D3): it scans the *builder prompt* for topic tokens that would leak into generated frameworks and flags them for generalisation. The seed is explicitly extensible and backed by a semantic LLM pass (builder-improve STEP 2). It is the opposite of a gate. *(Residual observation below.)*

Location	Hardcoded element	Verdict	Handling
ESG/sustainability disclosure-document seeds across `r6-discovery.ts`, `pipeline.ts:1166`, `primary-disclosure-check.ts`, `framework-v2/retrieval-targeting.ts:28`, `ranking.ts`	`"sustainability report"`, `"esg report"`, `"integrated report"`, `/\b(sustainab|esg|csr|...)/i`	REVIEWED-KEPT	These identify the **universal class of corporate non-financial disclosure documents** — the container where *any* ESG-adjacent framework's evidence (climate, nature, labour, data-privacy, AI-governance) is published. Discovery queries themselves are topic-driven: `discovery.ts:1430` builds queries from `framework.name` and explicitly instructs *"Do NOT generate generic sustainability/ESG queries unless that IS the topic."* Document-class seeds, not a topic gate.
`server/lib/analyzer.ts:102` `CSRD_TABLE_NOTATION_NOTE`	CSRD/ESRS time-horizon bullet-marker parsing hint	REVIEWED-KEPT	A **document-format** parsing aid for European CSRD/ESRS non-financial statements (how filled-circle horizon columns render in flattened PDFs). Applies to the *format* of a disclosure regardless of the framework's topic; harmless when such tables are absent. Analogous to the SEC-section logic. Not a topic gate. (Removing it

Location	Hardcoded element	Verdict	Handling
			would also be verdict-affecting — see §5.4.)
`server/lib/analyzer.ts:569` and `:725`	Scoring **system-prompt persona**: `"You are an expert ESG/governance analyst scoring corporate disclosures…"`	REVIEWED — **flagged as residual risk, not changed**	See §5.4. A mild ESG/governance *prior* in the scoring persona. The actual subject is injected at runtime (`Topic: ${topicDescription}` , and anti-inference rules derive from `frameworkDefinition.topicTerm`), so the persona is a weak prior, **not** a gate. It was **deliberately left unchanged** this pass because editing the live scoring prompt is a verdict-affecting runtime change and cannot be validated without a (prohibited) scoring batch.

5.3 Fixes applied this pass

None. This sweep found **no genuine hidden single-topic or single-company gate** in shared logic, prompts, or scoring that was not already handled by the earlier passes in §1–§3. Every topic/company literal encountered is one of: (a) a multi-topic illustrative prompt example, (b) a regression/motivation comment or test fixture, (c) a universal framework-pillar / disclosure-document / document-format construct that is topic-neutral by design, or (d) scoring infrastructure (model-provider names). The main app and Framework Builder V2 are **confirmed topic-agnostic at the gating level**: topic-specific behaviour is derived at runtime from the framework's own `topicTerm` / `topicDescription` / `topicSynonyms` / terminology and from the company record, not baked into shared code.

5.4 Residual risk (explicit)

- **Scoring-prompt persona lean (`analyzer.ts:569` , `:725`).** The scoring system prompt opens with *"You are an expert ESG/governance analyst…"*. For ESG-family topics (including AI **governance**, the first topic the product targeted) this is on-topic; for a hypothetical non-ESG framework it is a mild framing prior. It is **not** a gate — the real subject is supplied at runtime via

`topicDescription` and `framework.topicTerm` — but it is the one topic-leaning string left in the live scoring path. It was **not** changed in this pass because altering the scoring system prompt is a **verdict-affecting runtime change**: on genuinely borderline measures it could flip cells (yes→no / no→yes), which (i) is outside this task's build-time/advisory + agnosticism-refactor scope, and (ii) cannot be validated without running a scoring batch, which is prohibited here. Recommended future handling: generalise the persona to a neutral *"expert analyst scoring corporate disclosures against a structured assessment framework"* and validate verdict-stability on a frozen corpus before shipping. Logged here so the decision is explicit and owner-visible.

- **`generalisation-audit.ts` seed lexicon leans ESG/nature/climate** and does not list AI/digital-specific tokens. This only affects the *recall of the deterministic floor* of the builder-prompt leak detector, not scoring; the semantic LLM pass (builder-improve STEP 2) covers tokens the seed misses, and the seed comment already documents it as extensible. No functional gap for topic-agnosticism; noted for completeness.

5.5 Verification for this pass

- **No code changed in Part B** → `tsc --noEmit` server error count unchanged at **62** (pre-existing baseline on `main`; §4 recorded 61 at the time of the prior pass — the delta is pre-existing drift unrelated to this work, not introduced here). Client `tsc --noEmit` unchanged. No client code touched, so no `vite build` was required.
- The Issue 1 builder-stability code (Part A, commit `a8302a1`) is covered by its own tests (builder suite **370/370** passing) and added **0** new tsc errors.